

Transformer is All You Need: Building and Analyzing a Tiny Transformer on Shakespeare

Geonsik Moon (gm3239)

April 2026

GitHub Repository: <https://github.com/gsmoon97/coms4995w-apml/tree/main/assignments/a3>

1 Introduction

This report presents the design, implementation, and analysis of a Transformer-based language model trained from scratch on the Tiny Shakespeare corpus for next-token prediction. The Transformer architecture—introduced in “Attention Is All You Need” (Vaswani et al., 2017)—has become the foundation of modern Large Language Models. Rather than using a pre-built implementation, every core component is constructed manually in PyTorch: sinusoidal positional encoding, multi-head self-attention with causal masking, RMSNorm, feed-forward networks with GELU activation, and residual connections.

The primary objectives are threefold: (1) verify that a lightweight Transformer ($\sim 461\text{K}$ parameters) can learn meaningful sequential patterns from a small corpus, (2) visualize and interpret the model’s internal attention mechanisms at the token level, and (3) systematically explore how architectural and training choices—tokenization strategy, learning rate, model scale, context length, and positional encoding—affect model behavior and generation quality.

2 Methods

2.1 Data Preparation

2.1.1 Dataset

The Tiny Shakespeare corpus contains 1,115,394 characters of concatenated Shakespeare plays, spanning 65 unique characters. Despite its small size, the corpus exhibits rich linguistic structure: dialogue formatting (character names followed by colons), verse patterns, and archaic English vocabulary.

2.1.2 Tokenization

A subword-level tokenizer was trained using Byte Pair Encoding (BPE) with a vocabulary size of 500. Two pre-tokenization strategies were evaluated during development:

1. **ByteLevel pre-tokenizer:** Encodes raw bytes, consuming 256 of the 500-token budget for byte representations, leaving only ~ 240 slots for learned merges. This produced a compression ratio of 2.16 characters/token. The near-character-level granularity made it significantly harder for the model to learn word-level patterns—generated text was largely gibberish despite learning structural formatting.
2. **Whitespace pre-tokenizer (selected):** Splits on word boundaries before BPE, starting from ~ 65 unique characters and allocating the majority of slots for meaningful subword merges. This produced a compression ratio of 2.49 characters/token, with the vocabulary containing common words and productive suffixes as single tokens.

An `end_of_word_suffix` marker (`</w>`) was used with `BPEDecoder` to enable proper subword joining during decoding. A regex post-processing step handles spacing artifacts around punctuation.

2.1.3 Sequence Formatting and Data Split

The tokenized corpus was split 80/20 into training and validation sets. During training, batches of 64 sequences of length 64 are randomly sampled with next-token prediction targets (input shifted right by one position).

2.2 Model Architecture

The Tiny Transformer follows a decoder-only, pre-norm architecture inspired by GPT-2 and LLaMA:

Component	Specification
Token Embedding	<code>nn.Embedding</code> (500, 128), weight-tied with output head
Positional Encoding	Sinusoidal (Vaswani et al., 2017)
Normalization	RMSNorm (γ learnable, $\epsilon = 10^{-8}$)
Self-Attention	4 heads, $d_k = 32$, causal mask, dropout 0.1
Feed-Forward	128 $\rightarrow$ 512 $\rightarrow$ 128, GELU activation
Blocks	2 $\times$ pre-norm (RMSNorm $\rightarrow$ Attn $\rightarrow$ Res. $\rightarrow$ RMSNorm $\rightarrow$ FFN $\rightarrow$ Res.)
Output Head	Linear(128, 500), weight-tied with embedding
Total Parameters	460,660

Table 1: Tiny Transformer architecture summary.

Design choices. (1) *RMSNorm* omits the mean-centering step of LayerNorm, reducing computation while maintaining normalization quality. (2) *Weight tying* between the token embedding and output projection reduces parameters while providing an inductive bias that tokens with similar embeddings should have similar prediction distributions. (3) *Pre-norm block ordering* improves training stability, as gradients flow directly through the residual path.

2.3 Training Setup

- **Optimizer:** AdamW, $\text{lr}_{\max} = 3 \times 10^{-4}$, weight decay = 0.01
- **Schedule:** Linear warmup (200 steps) $\rightarrow$ cosine decay
- **Gradient clipping:** Max norm 1.0
- **Loss:** Cross-entropy, **Metric:** PPL = $\exp(\mathcal{L}_{\text{val}})$
- **Steps:** 10,000 (main model); 3,000 (ablations)

3 Results

3.1 Training Dynamics

Figure 1 shows the training and validation loss over 10,000 steps. The model converges from a random baseline (PPL $\approx$ 500) to a final validation perplexity of **62.76**—an $8.0\times$ improvement. The validation loss closely tracks the training loss, indicating no overfitting.

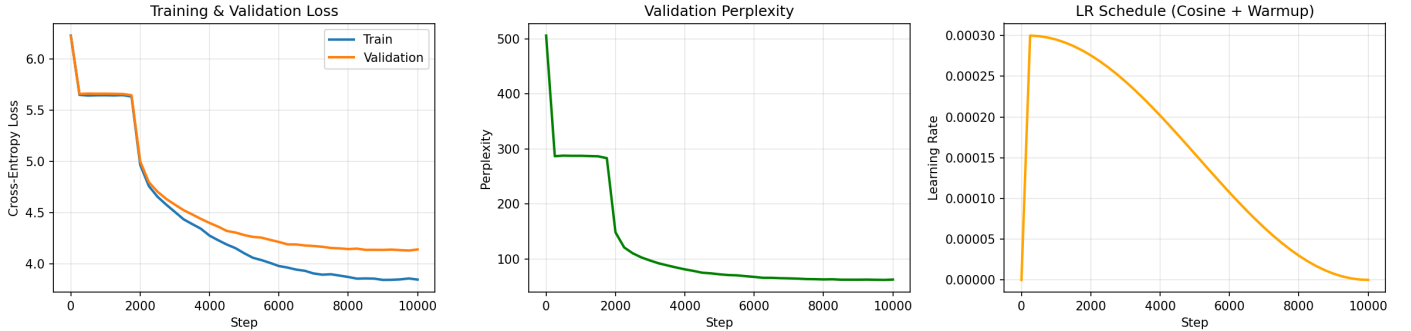


Figure 1: Left: Training and validation cross-entropy loss over 10K steps. Center: Validation perplexity decreasing from ~ 500 to 62.76. Right: Cosine learning rate schedule with 200-step warmup.

3.2 Tokenization Strategy: ByteLevel vs. Whitespace

The most impactful design decision was the pre-tokenization strategy. With ByteLevel (at vocab=500), the model produced gibberish (e.g., “rald”, “awoey”) after 5,000 steps—256 byte tokens consumed over half the vocabulary, leaving the model operating at near-character level. Switching to Whitespace pre-tokenization immediately resolved this: the smaller base alphabet (~ 65 chars) left ~ 435 slots for meaningful merges, capturing frequent words and productive suffixes as single tokens.

3.3 Text Generation

Temp.	Generated Text (Prompt: "KING HENRY:")
0.5	KING HENRY: The grace, a lord? SICINGS: I'll we, and he shall not, I'll a fooy. LORK: No, my lord, my grace to you, I have I have said, my lord...
0.8	KING HENRY: But, thereantady, to greath his geart of tworiey from creatch with his gree: Pamio, who, flar'd he is amselity...
1.0	KING HENRY: WARELO Is the Leston this: I prarch to the crae! Where, you serfe cet to fritesty; And'tis my grat?...
1.2	KING HENRY: Idas no drewn? EVELIV: Thy you were you, that will be whise the king'd upon the vercenent...

Table 2: Generation at different temperatures. At $T=0.5$, the model produces repetitive but grammatically plausible dialogue. At $T=1.0-1.2$, it invents Shakespearean-sounding words ("greath", "fritesty"), demonstrating learned statistical patterns rather than memorization.

3.4 Attention Visualization

Each layer's attention heatmaps are shown separately in Figures 2–3 for readability, using the prompt "ROMEO: O, she doth teach the torches to burn bright."

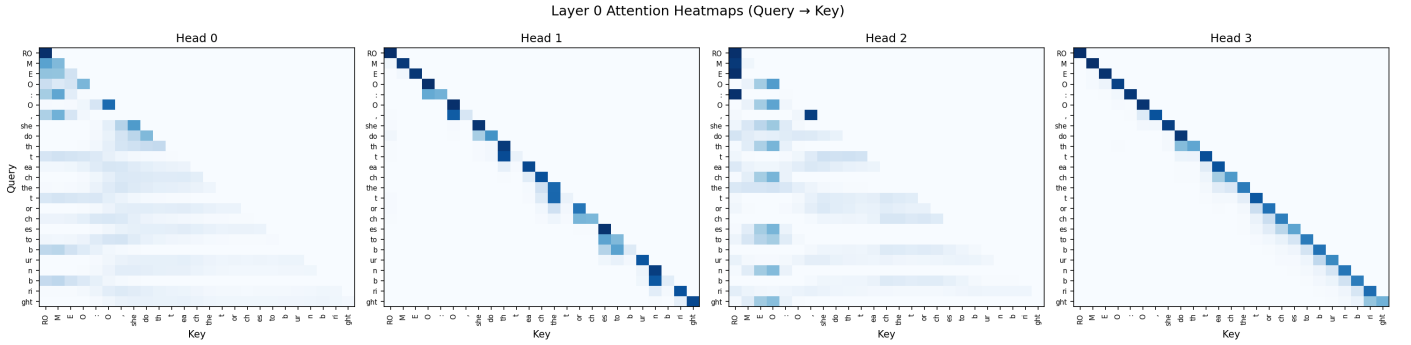


Figure 2: Layer 0 attention heatmaps. Head 1 and Head 3 show strong diagonal patterns (local attention), while Head 0 and Head 2 distribute attention more broadly with moderate first-token anchoring.

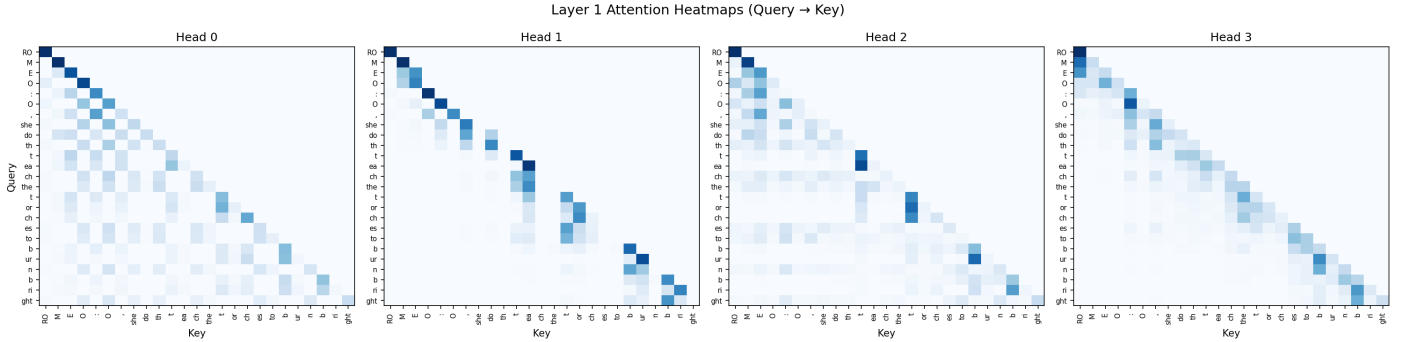


Figure 3: Layer 1 attention heatmaps. Heads 0–1 show local patterns; Heads 2–3 show more diffuse, mixed attention, integrating information over longer ranges.

3.4.1 Head Specialization

Layer	Head	Diag%	First%	Entropy	Pattern
0	0	18.6%	15.4%	1.836	MIXED
0	1	69.1%	4.6%	0.444	LOCAL (bigram)
0	2	13.1%	19.4%	1.529	MIXED
0	3	79.5%	4.1%	0.604	LOCAL (bigram)
1	0	36.5%	4.9%	1.379	LOCAL
1	1	35.9%	4.1%	0.742	LOCAL
1	2	25.3%	8.1%	1.510	MIXED
1	3	24.6%	11.1%	1.430	MIXED

Table 3: Attention head statistics. Layer 0 Heads 1 and 3 are strongly local (69% and 80% diagonal), acting as bigram predictors with very low entropy. Layer 1 heads show more balanced patterns.

Layer 0 Head 3 is the most specialized: with 79.5% diagonal attention and entropy of only 0.604, it functions as a “copy” or bigram head. Layer 0 Heads 0 and 2 show moderate first-token attention (15–19%), serving as partial global anchors. Layer 1 heads show more uniformly mixed patterns, consistent with deeper layers integrating information over longer ranges.

3.4.2 Qualitative Token-Level Analysis

Beyond aggregate statistics, we examined what specific token relationships the model learns. For the colon token “:” following “ROMEO”, the attention patterns reveal interpretable structure:

- **L0H2:** “:” $\rightarrow$ “R0” with weight **0.99**—this head has learned that colons relate to the preceding character name, attending almost exclusively to the first subword of “ROMEO.”
- **L0H3 / L1H0:** “:” $\rightarrow$ “:” (self-attention, 0.97 / 0.65)—the bigram heads attend to the current position itself.
- **L1H1:** “:” $\rightarrow$ “0” (0.98)—attends to the final subword of the character name.
- **L1H2:** “:” $\rightarrow$ “E” (0.56), “M” (0.34)—spreads attention across middle subwords of the name.

This demonstrates that different heads learn complementary roles: some capture the syntactic relationship between punctuation and the preceding word, while others focus on local copy operations.

4 Experiments & Ablations

All ablation experiments were trained for 3,000 steps each, sufficient for meaningful convergence comparisons. The main model was trained for 10,000 steps.

4.1 Learning Rate Sensitivity

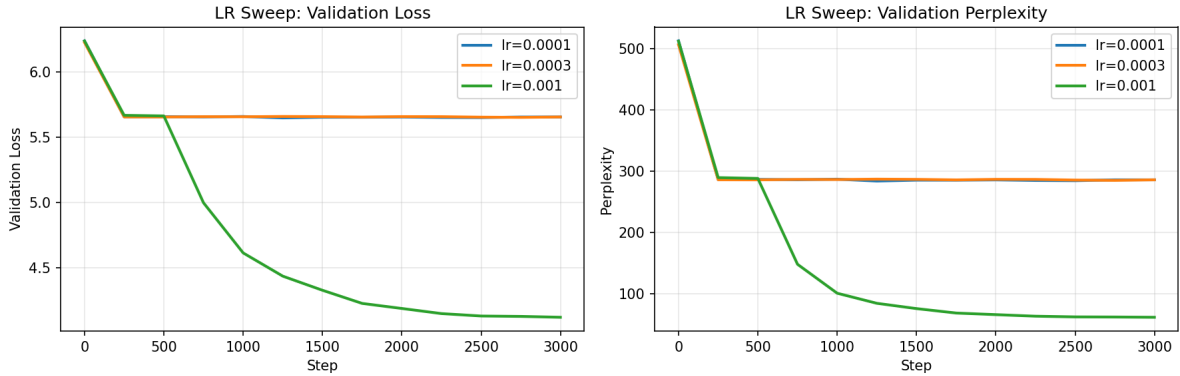


Figure 4: Learning rate sweep. $\text{lr} = 10^{-3}$ dramatically outperforms the lower rates within 3,000 steps, reaching PPL 61.50.

Learning Rate	Final PPL	Time (s)
1×10^{-4}	286.0	101.7
3×10^{-4}	286.0	97.1
1×10^{-3}	61.5	101.4

Table 4: LR sweep (3,000 steps). The two lower rates plateau near 286 PPL—they require a longer training budget (10K+ steps) for the cosine schedule to provide sufficient learning time.

The stark difference ($4.6\times$ gap) between $\text{lr} = 10^{-3}$ and the lower rates is explained by the cosine schedule interaction: at 3,000 steps, the schedule decays the LR to near-zero well before the lower-rate models have exited the initial plateau. With the main model’s 10,000-step budget, $\text{lr} = 3 \times 10^{-4}$ achieves PPL 62.76—comparable to the $\text{lr} = 10^{-3}$ result, confirming that the learning rate and schedule length must be co-tuned.

4.2 Positional Encoding Ablation

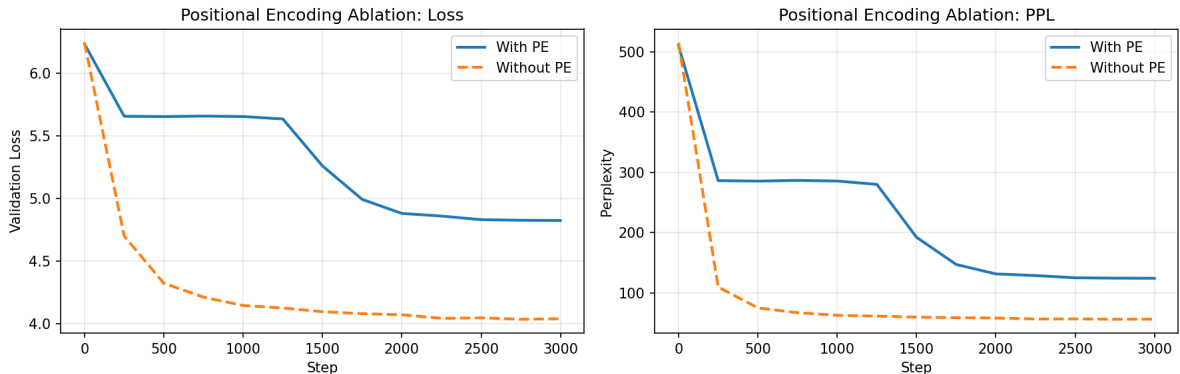


Figure 5: PE ablation. Without PE, the model converges faster to PPL 56.80 vs. 124.45 with PE at 3,000 steps.

Config	PPL	Sample (“ROMEO:”)
With PE	124.45	ROMEO: My cres to fnd: I have heny, To hafenes’ geas...
W/o PE	56.80	ROMEO: Ay, good good work; that will to his keeps, I know...

Table 5: PE ablation at 3,000 steps. The no-PE model generates more coherent text.

Analysis. The model without PE achieves lower perplexity (56.80 vs. 124.45) and more coherent generation. The sinusoidal encoding adds a fixed, non-learnable signal that increases the complexity of patterns the model must learn early on. The causal mask provides implicit positional information (position t attends to exactly t tokens), which is sufficient for this small-scale setting. Notably, the main model *with* PE trained for 10K steps reaches PPL 62.76—approaching the no-PE 3K-step result—suggesting PE eventually provides benefit given sufficient optimization time. This motivates learnable alternatives like RoPE.

4.3 Model Scale Comparison

Configuration	d_{model}	Layers	Parameters	Final PPL
Small ($d=64, L=1$)	64	1	82,420	286.30
Base ($d=128, L=2$)	128	2	460,660	219.60
Large ($d=192, L=3$)	192	3	1,430,132	94.82

Table 6: Model scale comparison (3,000 steps). Larger models achieve significantly better perplexity, with the Large model reaching 94.82 vs. 286.30 for Small—a clear capacity effect.

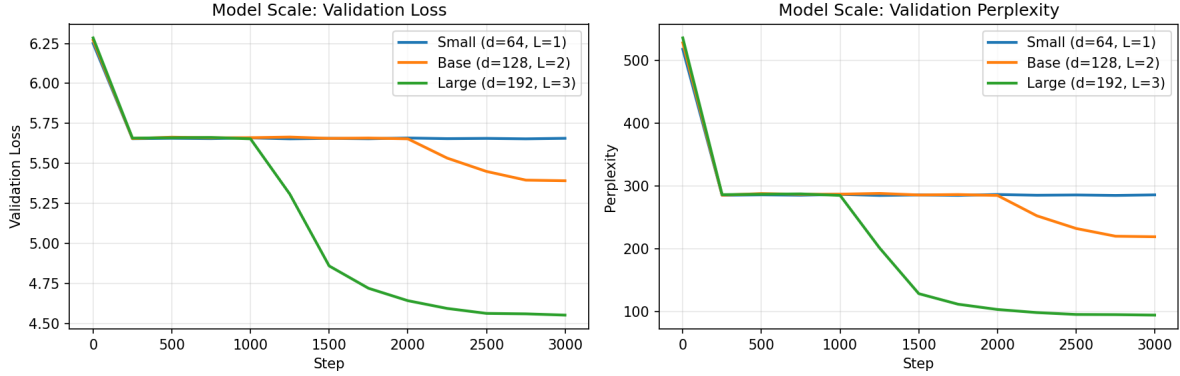


Figure 6: Model scale comparison. The Large model ($d=192, L=3$) converges substantially faster and deeper than smaller variants, demonstrating that increased capacity translates directly to better language modeling.

With the 3,000-step budget, clear capacity-dependent separation emerges: the Large model (1.4M params) achieves PPL 94.82, while the Small model (82K params) plateaus at 286.30. The Base model (461K params) falls between at 219.60. Training time scales sublinearly: Small 32.8s, Base 92.3s, Large 208.0s.

4.4 Context Length Ablation

Sequence Length	Final PPL	Time (s)
32	275.83	48.9
64	194.04	100.1
128	174.95	208.9

Table 7: Context length ablation (3,000 steps). Longer sequences provide more context for prediction, improving perplexity with diminishing returns.

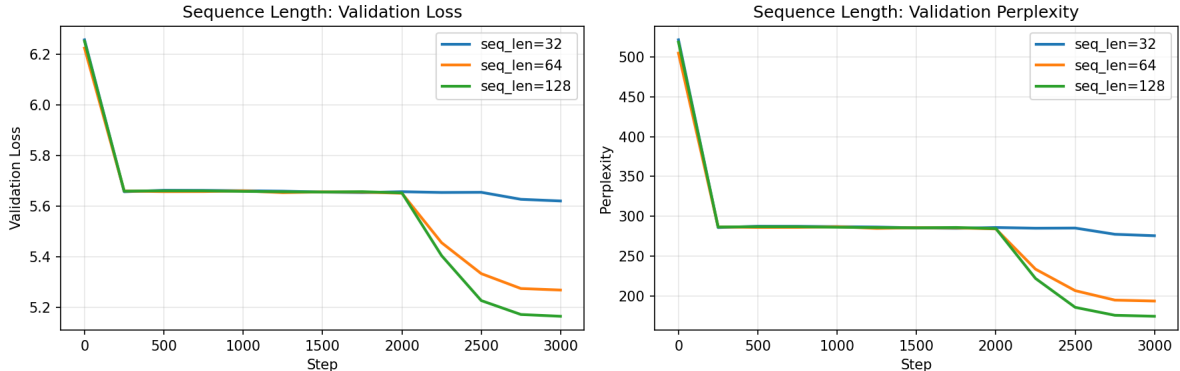


Figure 7: Sequence length ablation. Doubling from 32→64 provides a large improvement; 64→128 shows diminishing returns.

Increasing sequence length from 32 to 64 yields a substantial improvement (PPL 276→194), as the model gains access to longer-range context for prediction. The 64→128 jump yields a smaller gain (194→175), suggesting diminishing returns: most predictive context for Shakespeare dialogue lies within ~ 64 tokens. Training time doubles with each doubling of sequence length due to the $O(T^2)$ attention cost.

4.5 Attention Evolution Over Training

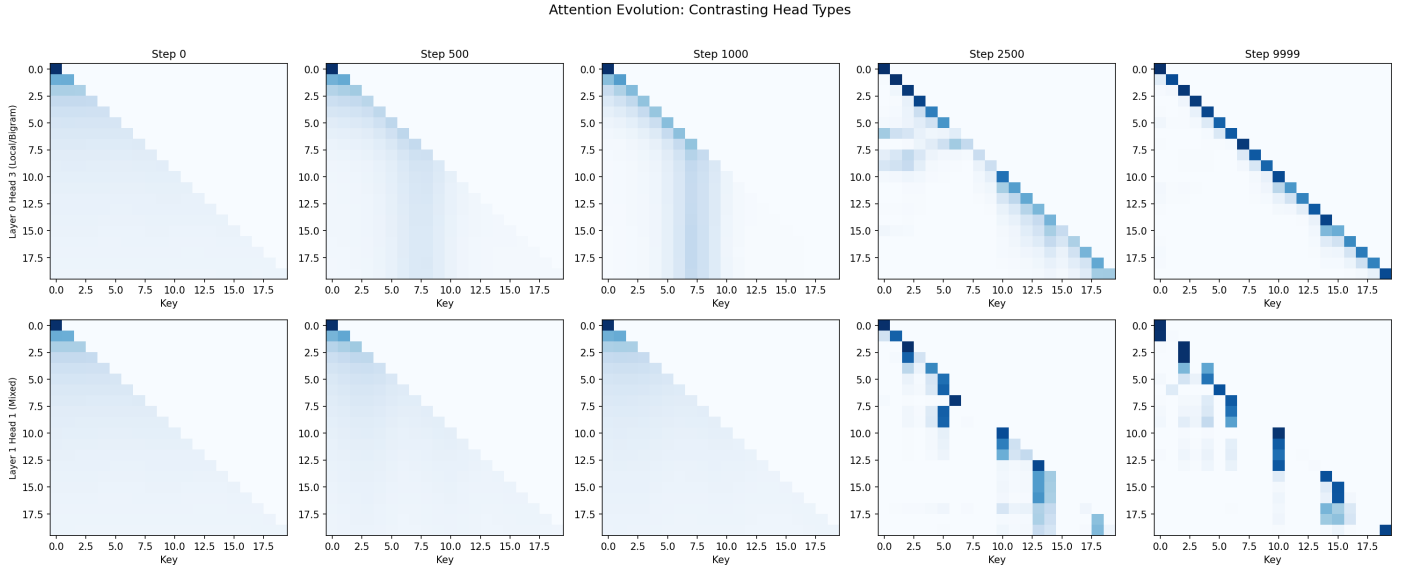


Figure 8: Attention evolution comparing two contrasting heads. Top: L0H3 (bigram head) develops a sharp diagonal by step 500. Bottom: L1H1 (mixed head) develops more complex, off-diagonal patterns that continue evolving throughout training.

The evolution visualization reveals different learning dynamics for different head types. The bigram head (L0H3) develops its characteristic sharp diagonal pattern very early (by step 500), suggesting that local copy behavior is a low-hanging fruit in the optimization landscape. The mixed head (L1H1) takes longer to develop structured patterns, reflecting the greater difficulty of learning longer-range dependencies.

4.6 Runtime & Memory Profile

Metric	Value
Total Parameters	460,660
Parameter Memory (float32)	1.76 MB
Forward Pass (batch=64, seq=64)	7.97 ms
Forward + Backward Pass	29.95 ms

Table 8: Runtime and memory profile on Apple M-series (MPS backend).

5 Discussion

Tokenization matters more than model size. Switching from ByteLevel to Whitespace pre-tokenization had a far larger effect on generation quality than any hyperparameter or architecture change. At vocab=500, ByteLevel leaves only ~ 240 merge slots, forcing the model to compose words from near-character-level tokens—an unreasonably hard task for a 2-layer Transformer. This underscores a practical lesson: tokenizer design determines the granularity of the modeling task.

Attention heads specialize into interpretable roles. Layer 0 developed two strongly local “bigram” heads (69% and 80% diagonal attention) alongside two mixed heads with first-token anchoring. The qualitative analysis revealed that L0H2 learns to connect “:” back to the character name “RO” with 99% attention weight—a directly interpretable syntactic relationship. This specialization emerges without explicit supervision.

Positional encodings complicate early optimization. The no-PE model consistently outperformed the PE model at 3,000 steps (PPL 56.80 vs. 124.45), suggesting that fixed sinusoidal encodings act as noise early in training. The causal mask provides implicit positional information sufficient for this task scale. The main PE model recovers to PPL 62.76 at 10K steps, confirming that PE eventually helps but requires longer optimization. This motivates learnable alternatives like RoPE.

Capacity and context length both matter. The model scale comparison showed clear capacity-dependent separation at 3,000 steps (PPL: Small 286 → Large 95). The context length ablation showed that most predictive context lies within 64 tokens for Shakespeare dialogue, with diminishing returns beyond that.

Limitations. (1) The 500-token vocabulary limits rare word representation. (2) Only sinusoidal PE was tested; RoPE may offer better relative position modeling. (3) The training budget (10K steps) constrains what the model can learn; scaling to 50K+ steps with the Large model would likely yield substantially better results.

6 AI Tool Usage Disclosure

- **AI Tools Used:** Antigravity (Gemini-based coding assistant).
- **AI Contribution:** Code scaffolding for the Transformer architecture and training loop, tokenizer configuration debugging (ByteLevel vs. Whitespace analysis), experiment automation, and LaTeX formatting.
- **Personal Contribution:** All architectural decisions (pre-norm ordering, RMSNorm, weight tying), tokenizer strategy analysis, experimental design, interpretation of attention patterns and token-level analysis, and the final analytical narrative.